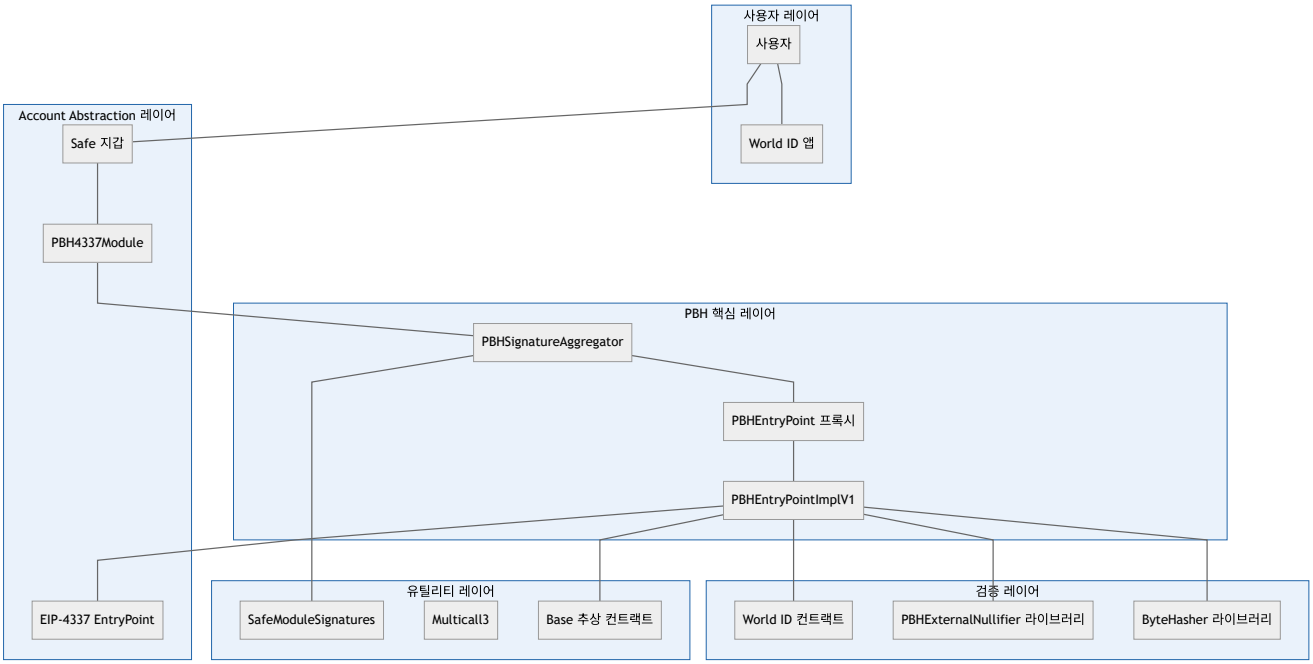
06. 스마트 컨트랙트 분석

개요

World Chain의 PBH(Priority Blockspace for Humans) 기능은 복잡한 온체인 스마트 컨트랙트 생태계를 통해 구현됩니다. 이 시스템은 World ID의 영지식 증명, EIP-4337 Account Abstraction, Safe 멀티시그 지갑, 그리고 커스텀 PBH 로직을 완벽하게 통합합니다.

전체 컨트랙트 아키텍처

핵심 컨트랙트 구조



주요 분석 대상 컨트랙트

1. **프록시 및 구현 계층**

- **PBHEntryPoint**: ERC1967 프록시 패턴 구현
- **PBHEntryPointImplV1**: 실제 PBH 로직 구현체

- **Base**: UUPS 업그레이드 가능한 추상 컨트랙트

2. **Account Abstraction 통합**

- **PBH4337Module**: Safe 지갑과 PBH 시스템 연결
- **PBHSignatureAggregator**: EIP-4337 서명 집계자

3. **검증 및 유틸리티**

- **PBHExternalNullifier**: 시간 기반 nullifier 관리
- **SafeModuleSignatures**: Safe 서명 처리 유틸리티
- **ByteHasher**: 필드 해시 생성 라이브러리

4. **외부 통합**

- **World ID 컨트랙트**: Semaphore 영지식 증명 검증
- **EIP-4337 EntryPoint**: 표준 Account Abstraction 엔트리포인트
- **Multicall3**: 배치 트랜잭션 실행

6.1 PBHEntryPoint 컨트랙트 분석

프록시 패턴 구현 방식

PBHEntryPoint 컨트랙트는 OpenZeppelin의 ERC1967Proxy를 상속하여 업그레이드 가능한 프록시 패턴을 구현합니다.

```
contract PBHEntryPoint is ERC1967Proxy {

constructor(address _logic, bytes memory  _data) payable  ERC1967Proxy(_logic, _

// 프록시 생성자에서는 로직을 구현하지 않음

// 모든 로직은 구현체(delegate)에서 처리

}
```

```
}
```

프록시 패턴의 핵심 특징

1. **투명한 프록시 (Transparent Proxy)**

- ERC1967 표준을 따라 구현체 주소를 특정 스토리지 슬롯에 저장
- 프록시와 구현체 간의 함수 충돌을 방지하는 메커니즘 제공

2. **위임 호출 (Delegatecall)**

- 모든 함수 호출이 구현체로 위임됨
- 프록시의 스토리지 컨텍스트에서 구현체 코드가 실행됨

3. **스토리지 분리**

- 프록시는 상태 변수를 선언하지 않음 (`DO NOT ADD MEMBERS HERE` 주석)
- 모든 상태는 구현체에서 관리됨

업그레이드 가능한 구조

UUPS (Universal Upgradeable Proxy Standard) 패턴

구현체인 PBHEntryPointImplV1은 UUPSUpgradeable을 상속하여 업그레이드 기능을 제공합니다:

```
abstract contract Base is Ownable2StepUpgradeable, UUPSUpgradeable {
function _authorizeUpgrade(address newImplementation)
internal virtual override onlyProxy onlyOwner {}
}
```

업그레이드 보안 메커니즘

1. **소유자 권한 제어**

- 오직 컨트랙트 소유자만 업그레이드 수행 가능
- 2단계 소유권 이전 프로세스로 실수 방지

2. **프록시 컨텍스트 검증**

- `onlyProxy` 모디파이어로 프록시를 통한 호출만 허용
- 직접 구현체 호출 방지

3. **스토리지 갭 (Storage Gap)**

- 향후 업그레이드 시 스토리지 충돌 방지를 위한 예약 공간
- `uint256[49] private __gap;` 으로 49개 슬롯 예약

초기화 과정과 설정 관리

초기화 메커니즘

PBH 시스템은 생성자 대신 초기화 함수를 사용합니다:

```
function initialize(
IWorldID _worldId,
IEntryPoint _entryPoint,
uint16 _numPbhPerMonth,
uint256 _pbhGasLimit,
address[] memory _authorizedBuilders,
address _owner
) external reinitializer(1)
```

초기화 매개변수 분석

1. **World ID 통합**

- `_worldId` : Semaphore 증명 검증을 위한 World ID 컨트랙트 주소
- 0 주소로 설정 시 오프체인 검증 모드로 동작

2. **EIP-4337 연동**

- `_entryPoint` : Account Abstraction EntryPoint 컨트랙트 주소
- PBH 트랜잭션을 일반 UserOperation으로 변환하여 처리

3. **PBH 정책 설정**

- `_numPbhPerMonth` : 월별 PBH 트랜잭션 허용 개수 (0-indexed)
- `_pbhGasLimit` : PBH 트랜잭션의 최대 가스 한도

4. **빌더 권한 관리**

- `_authorizedBuilders` : 승인된 블록 빌더 주소 목록
- 빌더만이 nullifier hash를 소비할 수 있음

초기화 검증 로직

```
// 필수 매개변수 검증
if (address(_entryPoint) == address(0)) {
revert AddressZero();
}

if (_numPbhPerMonth == 0) {
revert InvalidNumPbhPerMonth();
}

if (_authorizedBuilders.length == 0) {
```

```
revert InvalidAuthorizedBuilders();
}

// 가스 한도 검증
if (_pbhGasLimit == 0 || _pbhGasLimit > block.gaslimit) {
revert InvalidPBHGasLimit(_pbhGasLimit);
}
```

재초기화 방지

1. **구현체 생성자**

```
constructor() {
_disableInitializers();
}
```

- 구현체가 직접 초기화되는 것을 방지

2. **Reinitializer 패턴**

- `reinitializer(1)` 모디파이어로 버전별 초기화 관리
- 최대 255개의 초기화 버전 지원

설정 관리 시스템

동적 설정 변경

PBH 시스템은 운영 중에도 주요 설정을 변경할 수 있습니다:

1. **PBH 월별 한도 조정**

```
function  setNumPbhPerMonth(uint16 _numPbhPerMonth) external  onlyOwner {
numPbhPerMonth = _numPbhPerMonth;
emit NumPbhPerMonthSet(_numPbhPerMonth);
}
```

2. **World ID 주소 변경**

```
function  setWorldId(address _worldId) external  onlyOwner {
worldId = IWorldID(_worldId);
emit WorldIdSet(_worldId);
}
```

3. **가스 한도 조정**

```
function  setPBHGasLimit(uint256 _pbhGasLimit) external  onlyOwner {
pbhGasLimit = _pbhGasLimit;
emit PBHGasLimitSet(_pbhGasLimit);
}
```

빌더 권한 관리

```
function  addBuilder(address builder) external  onlyOwner {
authorizedBuilder[builder] = true;
emit BuilderAuthorized(builder);
```

```
}


function  removeBuilder(address builder) external  onlyOwner {

delete authorizedBuilder[builder];

emit BuilderDeauthorized(builder);

}
```

보안 고려사항

1. **접근 제어**

- 모든 관리 함수는 `onlyOwner` 모디파이어로 보호
- 프록시를 통한 호출만 허용하는 `onlyProxy` 검증

2. **재진입 공격 방지**

- `ReentrancyGuardTransient` 사용으로 재진입 공격 차단
- 임시 스토리지를 활용한 효율적인 가스 사용

3. **스토리지 충돌 방지**

- 업그레이드 시 스토리지 레이아웃 보존
- 예약된 스토리지 갭으로 향후 확장성 확보

이러한 프록시 패턴과 초기화 메커니즘을 통해 PBHEntryPoint는 안전하고 유연한 업그레이드 가능한 스마트 컨트랙트 시스템을 제공합니다.

6.2 PBHEntryPointImplV1 구현 분석

World ID 통합 방식

PBHEntryPointImplV1은 World ID 시스템과 긴밀하게 통합되어 Semaphore 영지식 증명을 검증합니다.

World ID 인터페이스 통합

```
/// @dev The World ID instance that will be used for verifying proofs

IWorldID public worldId;
```

World ID 통합의 핵심 구성 요소:

1. **유연한 검증 모드**

- 온체인 검증: `worldId` 주소가 설정된 경우
- 오프체인 검증: `worldId` 가 0 주소인 경우 빌더가 오프체인에서 검증

2. **Semaphore 증명 검증**

```
worldId.verifyProof(

pbhPayload.root, // Merkle tree root

signalHash, // 신호 해시

pbhPayload.nullifierHash, // nullifier 해시

pbhPayload.pbhExternalNullifier, // 외부 nullifier

pbhPayload.proof // 영지식 증명

);
```

PBH 페이로드 구조

```
struct PBHPayload {

uint256 root;  // World ID Merkle tree의 루트
```

```
uint256 pbhExternalNullifier;  // PBH 외부 nullifier

uint256 nullifierHash;  // 중복 방지용 nullifier 해시

uint256[8] proof;  // Semaphore 영지식 증명

}
```

각 필드의 역할:

1. **root**: World ID 시스템의 현재 Merkle tree 루트
- 사용자가 World ID에 등록되어 있음을 증명
- 주기적으로 업데이트되는 글로벌 상태
2. **pbhExternalNullifier**: 시간 기반 외부 nullifier
- 연도, 월, 논스를 인코딩한 값
- 월별 PBH 사용량 제한 구현
3. **nullifierHash**: 중복 사용 방지 해시
- 동일한 증명의 재사용을 방지
- 블록체인에 영구 저장되어 중복 검사
4. **proof**: Semaphore 영지식 증명
- 사용자의 신원을 노출하지 않고 World ID 멤버십 증명
- 8개 uint256 값으로 구성된 배열

EIP-4337 EntryPoint와의 연동 구현

Account Abstraction 통합 아키텍처

PBH 시스템은 EIP-4337 Account Abstraction과 완전히 통합되어 동작합니다:

```
/// @dev The EntryPoint where Aggregated PBH Bundles will be proxied to.
IEntryPoint public entryPoint;
```

집계된 UserOperation 처리

```
function handleAggregatedOps(
IEntryPoint.UserOpsPerAggregator[] calldata opsPerAggregator,
address payable beneficiary
) external virtual onlyProxy nonReentrant
```

처리 과정:

1. **배치 검증**

```
for (uint256 i = 0; i < opsPerAggregator.length; ++i) {
bytes32 hashedOps = keccak256(abi.encode(opsPerAggregator[i].userOps));
// 임시 스토리지에 해시 저장하여 중복 방지
assembly ("memory-safe") {
tstore(hashedOps, hashedOps)
}
}
```

2. **PBH 페이로드 디코딩**

```
PBHPayload[] memory pbhPayloads = abi.decode(opsPerAggregator[i].signature, (PBH
```

```
require(
pbhPayloads.length == opsPerAggregator[i].userOps.length,
InvalidAggregatedSignature(pbhPayloads.length, opsPerAggregator[i].userOps.lengt
);
```

3. **신호 해시 생성**

```
uint256 signalHash = abi.encodePacked(
sender, // 발신자 주소
opsPerAggregator[i].userOps[j].nonce, // 논스
opsPerAggregator[i].userOps[j].callData // 호출 데이터
).hashToField();
```

4. **EntryPoint로 위임**

```
entryPoint.handleAggregatedOps(opsPerAggregator, beneficiary);
```

UserOperation 해시 생성

```
function getUserOpHash(PackedUserOperation calldata userOp) public view virt
hash = keccak256(abi.encode(userOp.hash(), address(entryPoint), block.chainid));
}
```

- EIP-4337 표준에 따른 UserOperation 해시 생성
- 체인 ID와 EntryPoint 주소를 포함하여 재생 공격 방지

Nullifier Hash 관리 및 중복 방지 메커니즘

Nullifier Hash 저장 구조

```
/// @dev Whether a nullifier hash has been used already

mapping(uint256 nullifierHash => uint256 blockNumber) public nullifierHashes;
```

- 사용된 nullifier hash와 사용된 블록 번호를 매핑
- 0이 아닌 값은 이미 사용됨을 의미

중복 사용 검증

```
function  _verifyPbh(uint256 signalHash, PBHPayload memory  pbhPayload) internal

// nullifier hash 중복 사용 검사

if (nullifierHashes[pbhPayload.nullifierHash] != 0) {

revert InvalidNullifier(pbhPayload.nullifierHash, signalHash);

}

// 외부 nullifier 검증

PBHExternalNullifier.verify(pbhPayload.pbhExternalNullifier, numPbhPerMonth, sig

// World ID 증명 검증 (온체인 모드인 경우)

if (address(worldId) != address(0)) {

worldId.verifyProof(

pbhPayload.root, signalHash, pbhPayload.nullifierHash,

pbhPayload.pbhExternalNullifier, pbhPayload.proof

);
```

```
}

}
```

외부 Nullifier 검증 시스템

PBHExternalNullifier 라이브러리를 통한 시간 기반 제한:

```
library PBHExternalNullifier {

// 인코딩 형식:

// Bits 32-39: Year (연도)

// Bits 24-31: Month (월)

// Bits 8-23: Nonce (논스)

// Bits 0-7: Version (버전)

function  verify(uint256 externalNullifier, uint16 numPbhPerMonth, uint256 signa

(uint8 version, uint16 pbhNonce, uint8 month, uint16 year) = decode(externalNull

// 현재 연도/월과 일치하는지 검증

require(year == BokkyPooBahsDateTimeLibrary.getYear(block.timestamp));

require(month == BokkyPooBahsDateTimeLibrary.getMonth(block.timestamp));

// 논스가 월별 한도 내인지 검증

require(pbhNonce < numPbhPerMonth);

}

}
```

빌더 권한 기반 Nullifier 소비

```
function spendNullifierHashes(uint256[] calldata _nullifierHashes)
external virtual onlyProxy onlyBuilder {
for (uint256 i = 0; i < _nullifierHashes.length; ++i) {
nullifierHashes[_nullifierHashes[i]] = block.number;
}
emit NullifierHashesSpent(msg.sender, _nullifierHashes);
}
```

- 승인된 빌더만이 nullifier hash를 소비할 수 있음
- 블록 생성 후 해당 블록에 포함된 PBH 트랜잭션의 nullifier를 일괄 처리

Nullifier 상태 조회 기능

1. **첫 번째 미사용 Nullifier 찾기**

```
function getFirstUnspentNullifierHash(uint256[] calldata hashes)
public view virtual returns (int256) {
for (uint256 i = 0; i < hashes.length; ++i) {
if (nullifierHashes[i] == 0) {
return int256(i);
}
}
return -1; // 모두 사용됨
}
```

2. **모든 미사용 Nullifier 조회**

```
function getUnspentNullifierHashes(uint256[] calldata hashes)
public view virtual returns (uint256[] memory) {
// 미사용 nullifier의 인덱스 배열 반환
}
```

보안 메커니즘

임시 스토리지 활용

```
assembly ("memory-safe") {
if tload(hashedOps) {
mstore(0x00, 0x5e75ad06) // StorageCollision()
revert(0x1c, 0x04)
}
tstore(hashedOps, hashedOps)
}
```

- EIP-1153 임시 스토리지를 활용한 효율적인 중복 검사
- 트랜잭션 내에서만 유효한 임시 상태 관리

서명 검증 콜백

```
function validateSignaturesCallback(bytes32 hashedOps) external view virtual
assembly ("memory-safe") {
```

```
if  iszero(eq(tload(hashedOps), hashedOps)) {

mstore(0x00, 0xf5806179) // InvalidHashedOps()

revert(0x1c, 0x04)

}

}

}
```

- 집계된 서명의 유효성을 검증하는 콜백 함수
- 임시 스토리지의 해시 값과 비교하여 무결성 확인

이러한 구현을 통해 PBHEntryPointImplV1은 World ID의 영지식 증명 시스템과 EIP-4337 Account Abstraction을 완벽하게 통합하여, 안전하고 효율적인 PBH 트랜잭션 처리를 제공합니다.## 6.3 PBH

4337 UserOps 지원 분석

PBH4337Module과 PBHSignatureAggregator 분석

World Chain의 PBH 시스템은 EIP-4337 Account Abstraction과 완전히 통합되어 Safe 지갑을 통한 PBH 트랜잭션을 지원합니다.

PBH4337Module 아키텍처

PBH4337Module은 Safe4337Module을 확장하여 PBH 특화 기능을 제공합니다:

```
contract PBHSafe4337Module is Safe4337Module {

/// @notice The PBH Signature Aggregator address.

address public  immutable PBH_SIGNATURE_AGGREGATOR;

/// @notice The PBH Nonce Key.
```

```
/// @dev This key is used to identify a PBH user operation.
uint40 public immutable PBH_NONCE_KEY; // bytes5("pbhtx")
}
```

핵심 구성 요소:

1. **PBH 논스 키**: `bytes5("pbhtx")` 로 PBH 트랜잭션 식별
2. **서명 집계자**: PBH 증명 검증을 담당하는 집계자 주소
3. **증명 데이터 길이**: 352바이트 고정 크기의 인코딩된 증명 데이터

PBH 트랜잭션 식별 메커니즘

```
function _validateSignatures(PackedUserOperation calldata userOp)
internal view override returns (uint256 validationData) {
// 논스에서 키 추출 (상위 192비트 중 첫 40비트)
uint192 key = uint192(userOp.nonce >> 64);
bool isPBH = (key >> 152) == PBH_NONCE_KEY;
if (isPBH && validSignature) {
authorizer = PBH_SIGNATURE_AGGREGATOR;
} else {
authorizer = validSignature ? address(0) : address(1);
}
}
```

PBH 트랜잭션 식별 과정:

1. **논스 구조 분석**: EIP-4337 논스의 상위 비트에서 키 추출

2. **PBH 키 매칭**: 추출된 키가 `bytes5("pbhtx")` 와 일치하는지 확인

3. **권한자 설정**: PBH 트랜잭션인 경우 PBH_SIGNATURE_AGGREGATOR를 권한자로 설정

UserOperation 처리 방식과 번들링 과정

서명 구조 분석

PBH UserOperation의 서명은 다음과 같이 구성됩니다:

```
[12 bytes: timestamp] + [variable: Safe signatures] + [352 bytes: PBH proof]
```

구성 요소별 분석:

1. **타임스탬프 (12바이트)**

- `validAfter` (6바이트): 유효 시작 시간
- `validUntil` (6바이트): 유효 종료 시간

2. **Safe 서명 (가변 길이)**

- 임계값에 따른 ECDSA 서명들
- 컨트랙트 서명 포함 가능

3. **PBH 증명 (352바이트)**

- 인코딩된 PBHPayload 구조체
- World ID Semaphore 증명 데이터

서명 검증 및 증명 추출

```
function _validateSignatures(PackedUserOperation calldata userOp) internal vi

uint256 threshold = ISafe(payable(userOp.sender)).getThreshold();

// 기본 서명 길이 계산
```

```
uint256 expectedLength = TIMESTAMP_BYTES +

SafeModuleSignatures.signatureLength(userOp.signature[TIMESTAMP_BYTES:], thresho

// 증명 데이터가 첨부된 경우 (번들러 시뮬레이션)

if (isPBH && userOp.signature.length > expectedLength) {

if (userOp.signature.length - expectedLength != ENCODED_PROOF_BYTES) {

revert InvalidProofSize();

}

// 증명 데이터 제거 후 서명 검증

signatures = userOp.signature[TIMESTAMP_BYTES:expectedLength];

}

}
```

SafeModuleSignatures 라이브러리

증명 데이터 추출을 위한 유틸리티 함수:

```
library SafeModuleSignatures {

function  extractProof(bytes calldata  signatures, uint256 threshold)

internal  pure  returns (bytes memory  userOperationSignature, bytes memory  pro

uint256 length = TIMESTAMP_BYTES + signatureLength(signatures[TIMESTAMP_BYTES:],

// 전체 길이 검증

require(signatures.length == length + PROOF_DATA_LENGTH);

// 증명 데이터와 서명 분리

proofData = signatures[length:length + PROOF_DATA_LENGTH];
```

```
userOperationSignature = signatures[0:length];

}

}
```

서명 집계 및 검증 메커니즘

PBHSignatureAggregator 역할

PBHSignatureAggregator는 IAggregator 인터페이스를 구현하여 EIP-4337 집계자 역할을 수행합니다:

```
contract PBHSignatureAggregator is IAggregator {

/// @notice The PBHVerifier contract.

IPBHEntryPoint public  immutable pbhEntryPoint;

/// @notice The WorldID contract.

IWorldID public  immutable worldID;

}
```

개별 UserOperation 검증

```
function  validateUserOpSignature(PackedUserOperation calldata  userOp)

external  view  returns (bytes memory  sigForUserOp) {

// 1. 증명 데이터 추출

bytes memory proofData;

(sigForUserOp, proofData) = SafeModuleSignatures.extractProof(

userOp.signature,
```

```
ISafe(payable(userOp.sender)).getThreshold()

);

// 2. PBH 페이로드 디코딩

IPBHEntryPoint.PBHPayload memory pbhPayload = abi.decode(proofData, (IPBHEntryPo

// 3. 신호 해시 생성

uint256 signalHash = abi.encodePacked(

userOp.sender,

userOp.nonce,

userOp.callData

).hashToField();

// 4. PBH 검증

pbhEntryPoint.verifyPbh(signalHash, pbhPayload);

// 5. 오프체인 World ID 검증 (필요시)

if (address(pbhEntryPoint.worldId()) == address(0)) {

worldID.verifyProof(

pbhPayload.root, signalHash, pbhPayload.nullifierHash,

pbhPayload.pbhExternalNullifier, pbhPayload.proof

);

}

}
```

검증 단계별 분석:

1. **증명 추출**: 서명에서 PBH 증명 데이터 분리

2. **페이로드 디코딩**: 352바이트 증명을 PBHPayload 구조체로 변환

3. **신호 해시 생성**: 발신자, 논스, 호출 데이터로부터 고유 해시 생성

4. **PBH 검증**: PBHEntryPoint를 통한 증명 유효성 검사

5. **World ID 검증**: 오프체인 모드에서 직접 World ID 검증

서명 집계 프로세스

```
function aggregateSignatures(PackedUserOperation[] calldata userOps)
external view returns (bytes memory aggregatedSignature) {
IPBHEntryPoint.PBHPayload[] memory pbhPayloads = new IPBHEntryPoint.PBHPayload[]
// 각 UserOperation에서 PBH 페이로드 추출
for (uint256 i = 0; i < userOps.length; ++i) {
(, bytes memory proofData) = SafeModuleSignatures.extractProof(
userOps[i].signature,
ISafe(payable(userOps[i].sender)).getThreshold()
);
pbhPayloads[i] = abi.decode(proofData, (IPBHEntryPoint.PBHPayload));
}
// 모든 페이로드를 하나의 집계된 서명으로 인코딩
aggregatedSignature = abi.encode(pbhPayloads);
}
```

집계된 서명 검증

```
function  validateSignatures(PackedUserOperation[] calldata  userOps, bytes call

bytes memory encoded = abi.encode(userOps);

pbhEntryPoint.validateSignaturesCallback(keccak256(encoded));

}
```

- UserOperation 배열을 해시하여 무결성 검증
- PBHEntryPoint의 임시 스토리지와 비교하여 일치성 확인

번들링 워크플로우

1. 사용자 측 준비

1. **PBH 증명 생성**: World ID를 통한 Semaphore 증명 생성
2. **서명 구성**: Safe 서명 + PBH 증명을 하나의 서명으로 결합
3. **논스 설정**: PBH 키(`bytes5("pbhtx")`)를 포함한 논스 생성

2. 번들러 측 처리

1. **시뮬레이션**: 증명이 포함된 전체 서명으로 시뮬레이션 수행
2. **증명 검증**: `validateUserOpSignature` 를 통한 개별 증명 검증
3. **서명 정리**: 증명 데이터 제거 후 실행용 서명 생성
4. **집계**: 여러 PBH UserOperation의 증명을 하나로 집계

3. 실행 단계

1. **집계된 처리**: `handleAggregatedOps` 를 통한 배치 실행
2. **증명 검증**: 각 UserOperation의 PBH 증명 개별 검증
3. **EntryPoint 위임**: 검증 완료 후 표준 EIP-4337 EntryPoint로 위임

4. **Nullifier 소비**: 블록 생성 후 사용된 nullifier hash 일괄 처리

보안 및 효율성 고려사항

가스 최적화

1. **증명 데이터 분리**: 실행 시에는 증명 데이터 제외하여 가스 절약
2. **배치 처리**: 여러 PBH 트랜잭션을 한 번에 처리하여 효율성 향상
3. **임시 스토리지**: EIP-1153을 활용한 효율적인 상태 관리

보안 메커니즘

1. **서명 길이 검증**: 악의적인 패딩 방지를 위한 정확한 길이 검사
2. **증명 무결성**: 352바이트 고정 크기로 증명 데이터 무결성 보장
3. **권한 분리**: 집계자와 검증자 역할 분리로 보안 강화

이러한 구조를 통해 World Chain은 EIP-4337 Account Abstraction과 완벽하게 통합된 PBH 시스템을 제공하며, Safe 지갑 사용자들이 World ID 증명을 통해 우선순위 블록스페이스에 접근할 수 있도록 합니다.

6.4 유틸리티 라이브러리 및 지원 컨트랙트 분석

ByteHasher 라이브러리

ByteHasher는 바이트 데이터를 Semaphore 호환 필드 요소로 변환하는 핵심 유틸리티입니다.

```
library ByteHasher {
function hashToField(bytes memory value) internal pure returns (uint256) {
return uint256(keccak256(value)) >> 8;
}
```

```
}
```

필드 해시 생성 원리

1. **Keccak256 해시**: 입력 바이트를 표준 Ethereum 해시 함수로 처리
2. **8비트 우측 시프트**: 결과를 `>> 8` 로 시프트하여 Semaphore 필드 크기에 맞춤
3. **필드 호환성**: BN254 곡선의 스칼라 필드 크기(254비트) 내로 제한

사용 사례

```
// PBH 신호 해시 생성
uint256 signalHash = abi.encodePacked(
sender, // 발신자 주소 (20바이트)
userOp.nonce, // 논스 (32바이트)
userOp.callData // 호출 데이터 (가변 길이)
).hashToField();
```

SafeModuleSignatures 라이브러리

Safe 지갑의 복잡한 서명 구조를 처리하는 전문 라이브러리입니다.

서명 길이 계산

```
function signatureLength(bytes calldata signatures, uint256 threshold)
internal pure returns (uint256 expectedLength) {
expectedLength = ECDSA_SIGNATURE_LENGTH * threshold;  // 기본 ECDSA 서명들
// 컨트랙트 서명 처리
```

```
for (uint256 i = 0; i < threshold; ++i) {
uint256 signaturePos = i * ECDSA_SIGNATURE_LENGTH;
uint8 signatureType = uint8(signatures[signaturePos + 0x40]);
if (signatureType == 0) { // 컨트랙트 서명
uint256 signatureOffset = uint256(bytes32(signatures[signaturePos + 0x20:]));
uint256 length = uint256(bytes32(signatures[signatureOffset:]));
expectedLength += 0x20 + length;  // 길이 필드 + 실제 데이터
}
}
}
```

Safe 서명 타입 분석

1. **ECDSA 서명 (타입 0)**: 표준 65바이트 ECDSA 서명
2. **컨트랙트 서명 (타입 1)**: EIP-1271 기반 컨트랙트 서명
3. **승인된 해시 (타입 2)**: 사전 승인된 트랜잭션 해시

PBH 증명 추출

```
function  extractProof(bytes calldata  signatures, uint256 threshold)
internal  pure  returns (bytes memory  userOperationSignature, bytes memory  pro
// 최소 길이 검증: 타임스탬프 + ECDSA + 증명 데이터
require(signatures.length >= TIMESTAMP_BYTES + ECDSA_SIGNATURE_LENGTH + PROOF_DA
uint256 length = TIMESTAMP_BYTES + signatureLength(signatures[TIMESTAMP_BYTES:],
// 정확한 길이 검증
```

```
require(signatures.length == length + PROOF_DATA_LENGTH);

// 데이터 분리

proofData = signatures[length:length + PROOF_DATA_LENGTH];

userOperationSignature = signatures[0:length];

}
```

PBHExternalNullifier 라이브러리 심화 분석

인코딩 구조 상세

```
256비트 External Nullifier 구조:

┌──────────────────┬──────────┬──────────┬──────────┬─────────┐

| Reserved | Year | Month | Nonce | Version |

| (216 bits) | (8 bits) | (8 bits) | (16 bits)| (8 bits)|

| Bits 40-255 | Bits 32-39|Bits 24-31|Bits 8-23 | Bits 0-7|

└──────────────────┴──────────┴──────────┴──────────┴─────────┘
```

인코딩 및 디코딩 함수

```
function  encode(uint8 version, uint16 pbhNonce, uint8 month, uint16 year)

internal  pure  returns (uint256) {

require(month > 0 && month < 13, InvalidExternalNullifierMonth());

return (uint256(year) << 32) | (uint256(month) << 24) |

(uint256(pbhNonce) << 8) | uint256(version);
```

```
}


function  decode(uint256 externalNullifier)

internal  pure  returns (uint8 version, uint16 pbhNonce, uint8 month, uint16 yea

year = uint16(externalNullifier >> 32);

month = uint8((externalNullifier >> 24) & 0xFF);

pbhNonce = uint16((externalNullifier >> 8) & 0xFFFF);

version = uint8(externalNullifier & 0xFF);

}
```

시간 기반 검증 로직

```
function  verify(uint256 externalNullifier, uint16 numPbhPerMonth, uint256 signa

// 1. 크기 검증 (48비트 이내)

require(externalNullifier <= type(uint48).max, "Leading zeros");

// 2. 구성 요소 추출

(uint8 version, uint16 pbhNonce, uint8 month, uint16 year) = decode(externalNull

// 3. 버전 검증

require(version == V1, "Invalid Version");

// 4. 현재 시간과 비교

require(year == BokkyPooBahsDateTimeLibrary.getYear(block.timestamp), "Invalid Y

require(month == BokkyPooBahsDateTimeLibrary.getMonth(block.timestamp), "Invalid

// 5. 논스 범위 검증

require(pbhNonce < numPbhPerMonth, "Invalid PBH Nonce");
```

```
}
```

Base 추상 컨트랙트 심화 분석

Base 컨트랙트는 모든 업그레이드 가능한 PBH 컨트랙트의 기반을 제공합니다.

다중 상속 구조

```
abstract contract Base is Ownable2StepUpgradeable, UUPSUpgradeable {
function __Base_init(address owner) internal virtual onlyInitializing {
__Ownable_init(owner); // 소유권 관리 초기화
__UUPSUpgradeable_init(); // UUPS 프록시 초기화
}
}
```

업그레이드 권한 제어

```
function _authorizeUpgrade(address newImplementation)
internal virtual override onlyProxy onlyOwner {
// 소유자만이 프록시를 통해 업그레이드 가능
// newImplementation 매개변수는 향후 검증 로직 확장 가능
}
```

스토리지 갭 관리

```
uint256[49] private __gap;
```

- **목적**: 향후 업그레이드 시 새로운 상태 변수 추가 공간 확보
- **크기**: 49개 슬롯 (Base 컨트랙트용 1개 슬롯 + 예비 49개)
- **중요성**: 상속 체인에서 스토리지 충돌 방지

6.5 완전한 트랜잭션 생명주기 분석

1단계: 사용자 준비 과정

World ID 증명 생성

```
// 클라이언트 측 의사코드
const proof = await worldId.generateProof({
signal: signalHash, // 트랜잭션 고유 식별자
externalNullifier: pbhExternalNullifier, // 시간 기반 nullifier
nullifierHash: computedNullifierHash // 중복 방지 해시
});
```

PBH UserOperation 구성

```
// Safe 서명 + PBH 증명 결합
bytes memory signature = abi.encodePacked(
uint48(validAfter), // 6바이트: 유효 시작 시간
uint48(validUntil), // 6바이트: 유효 종료 시간
```

```
safeSignatures, // 가변 길이: Safe 멀티시그 서명들
encodedPBHPayload // 352바이트: PBH 증명 데이터
);


PackedUserOperation memory userOp = PackedUserOperation({
sender: address(safe),
nonce: (uint256(PBH_NONCE_KEY) << 216) | nonce, // PBH 식별 논스
initCode: "",
callData: multicallData,
accountGasLimits: gasLimits,
preVerificationGas: preVerificationGas,
gasFees: gasFees,
paymasterAndData: "",
signature: signature
});
```

2단계: 번들러 검증 과정

개별 UserOperation 검증

```
function validateUserOpSignature(PackedUserOperation calldata userOp)
external view returns (bytes memory sigForUserOp) {
// 1. 서명에서 증명 데이터 분리
(sigForUserOp, bytes memory proofData) = SafeModuleSignatures.extractProof(
```

```
userOp.signature,

ISafe(payable(userOp.sender)).getThreshold()

);

// 2. PBH 페이로드 디코딩

IPBHEntryPoint.PBHPayload memory pbhPayload = abi.decode(proofData, (IPBHEntryPo

// 3. 신호 해시 재생성

uint256 signalHash = abi.encodePacked(

userOp.sender, userOp.nonce, userOp.callData

).hashToField();

// 4. PBH 증명 검증

pbhEntryPoint.verifyPbh(signalHash, pbhPayload);

// 5. 오프체인 World ID 검증 (필요시)

if (address(pbhEntryPoint.worldId()) == address(0)) {

worldID.verifyProof(

pbhPayload.root, signalHash, pbhPayload.nullifierHash,

pbhPayload.pbhExternalNullifier, pbhPayload.proof

);

}

}
```

서명 집계 과정

```
function  aggregateSignatures(PackedUserOperation[] calldata  userOps)
```

```
external view returns (bytes memory aggregatedSignature) {
IPBHEntryPoint.PBHPayload[] memory pbhPayloads = new IPBHEntryPoint.PBHPayload[]
for (uint256 i = 0; i < userOps.length; ++i) {
// 각 UserOperation에서 PBH 증명 추출
(, bytes memory proofData) = SafeModuleSignatures.extractProof(
userOps[i].signature,
ISafe(payable(userOps[i].sender)).getThreshold()
);
pbhPayloads[i] = abi.decode(proofData, (IPBHEntryPoint.PBHPayload));
}
// 모든 증명을 하나의 집계된 서명으로 인코딩
aggregatedSignature = abi.encode(pbhPayloads);
}
```

3단계: Safe 지갑 검증

PBH4337Module 서명 검증

```
function _validateSignatures(PackedUserOperation calldata userOp)
internal view override returns (uint256 validationData) {
// 1. PBH 트랜잭션 식별
uint192 key = uint192(userOp.nonce >> 64);
bool isPBH = (key >> 152) == PBH_NONCE_KEY;
// 2. 서명 길이 계산 및 검증
```

```
uint256 threshold = ISafe(payable(userOp.sender)).getThreshold();

uint256 expectedLength = TIMESTAMP_BYTES +

SafeModuleSignatures.signatureLength(userOp.signature[TIMESTAMP_BYTES:], thresho

// 3. PBH 증명 데이터 처리

bytes calldata signatures;

if (isPBH && userOp.signature.length > expectedLength) {

// 번들러 시뮬레이션: 증명 데이터 제거

require(userOp.signature.length - expectedLength == ENCODED_PROOF_BYTES);

signatures = userOp.signature[TIMESTAMP_BYTES:expectedLength];

} else {

// 실제 실행: 증명 데이터 없음

signatures = userOp.signature[TIMESTAMP_BYTES:];

}

// 4. Safe 서명 검증

(bytes memory operationData, uint48 validAfter, uint48 validUntil,) = _getSafeOp

bool validSignature = _checkSignaturesLength(signatures, threshold);

try ISafe(payable(userOp.sender)).checkSignatures(

keccak256(operationData), operationData, signatures

) {} catch {

validSignature = false;

}

// 5. 권한자 설정

address authorizer;
```

```
if (isPBH && validSignature) {
authorizer = PBH_SIGNATURE_AGGREGATOR;  // PBH 집계자로 위임
} else {
authorizer = validSignature ? address(0) : address(1);
}
return  _packValidationData(ValidationData(authorizer, validAfter, validUntil));
}
```

4단계: PBH 시스템 검증

집계된 UserOperation 처리

```
function  handleAggregatedOps(
IEntryPoint.UserOpsPerAggregator[] calldata  opsPerAggregator,
address payable  beneficiary
) external  virtual  onlyProxy  nonReentrant {
for (uint256 i = 0; i < opsPerAggregator.length; ++i) {
// 1. 임시 스토리지에 해시 저장 (중복 방지)
bytes32 hashedOps = keccak256(abi.encode(opsPerAggregator[i].userOps));
assembly ("memory-safe") {
if  tload(hashedOps) {
revert(0x1c, 0x04) // StorageCollision()
}
tstore(hashedOps, hashedOps)
```

```
}

// 2. 집계된 서명에서 PBH 페이로드 추출

PBHPayload[] memory pbhPayloads = abi.decode(

opsPerAggregator[i].signature, (PBHPayload[])

);

// 3. 페이로드 개수 검증

require(pbhPayloads.length == opsPerAggregator[i].userOps.length);

// 4. 각 UserOperation 개별 검증

for (uint256 j = 0; j < pbhPayloads.length; ++j) {

address sender = opsPerAggregator[i].userOps[j].sender;

// 신호 해시 재생성

uint256 signalHash = abi.encodePacked(

sender,

opsPerAggregator[i].userOps[j].nonce,

opsPerAggregator[i].userOps[j].callData

).hashToField();

// PBH 증명 검증

_verifyPbh(signalHash, pbhPayloads[j]);

// 이벤트 발생

bytes32 userOpHash = getUserOpHash(opsPerAggregator[i].userOps[j]);

emit PBH(sender, userOpHash, pbhPayloads[j]);

}

}
```

```
// 5. 표준 EntryPoint로 위임
entryPoint.handleAggregatedOps(opsPerAggregator, beneficiary);
}
```

PBH 증명 검증 세부 과정

```
function  _verifyPbh(uint256 signalHash, PBHPayload memory  pbhPayload) internal
// 1. Nullifier 중복 사용 검사
if (nullifierHashes[pbhPayload.nullifierHash] != 0) {
revert InvalidNullifier(pbhPayload.nullifierHash, signalHash);
}
// 2. 외부 nullifier 검증 (시간 및 논스 제한)
PBHExternalNullifier.verify(pbhPayload.pbhExternalNullifier, numPbhPerMonth, sig
// 3. World ID 증명 검증 (온체인 모드)
if (address(worldId) != address(0)) {
worldId.verifyProof(
pbhPayload.root, // Merkle tree 루트
signalHash, // 신호 해시
pbhPayload.nullifierHash, // Nullifier 해시
pbhPayload.pbhExternalNullifier, // 외부 nullifier
pbhPayload.proof // Semaphore 증명
);
}
```

```
// 오프체인 모드에서는 번들러가 이미 검증했다고 가정
}
```

5단계: 실행 및 후처리

EIP-4337 EntryPoint 실행

표준 Account Abstraction 프로토콜에 따라 UserOperation들이 실행됩니다:

1. **가스 계산**: 각 UserOperation의 가스 사용량 계산
2. **실행**: Safe 지갑을 통한 실제 트랜잭션 실행
3. **가스 환불**: 번들러에게 가스 비용 지불
4. **이벤트 발생**: UserOperationEvent 발생

Nullifier Hash 소비

블록 생성 후 승인된 빌더가 사용된 nullifier hash들을 일괄 처리합니다:

```
function spendNullifierHashes(uint256[] calldata _nullifierHashes)
external virtual onlyProxy onlyBuilder {
for (uint256 i = 0; i < _nullifierHashes.length; ++i) {
nullifierHashes[_nullifierHashes[i]] = block.number; // 사용된 블록 번호 기록
}
emit NullifierHashesSpent(msg.sender, _nullifierHashes);
}
```

6.6 고급 보안 메커니즘 및 최적화

임시 스토리지 활용 (EIP-1153)

```
assembly ("memory-safe") {
// 중복 검사
if  tload(hashedOps) {
mstore(0x00, 0x5e75ad06) // StorageCollision()
revert(0x1c, 0x04)
}
// 임시 저장
tstore(hashedOps, hashedOps)
}
```

장점:

- 트랜잭션 내에서만 유효한 상태 관리
- 영구 스토리지 사용 없이 중복 방지
- 가스 효율성 향상

재진입 공격 방지

```
contract PBHEntryPointImplV1 is ReentrancyGuardTransient {
function  handleAggregatedOps(...) external  nonReentrant {
// 재진입 방지된 실행
}
}
```

특징:

- `ReentrancyGuardTransient` 사용으로 임시 스토리지 활용
- 기존 `ReentrancyGuard` 보다 가스 효율적
- 트랜잭션 종료 시 자동 정리

서명 길이 검증

```
function _checkSignaturesLength(bytes calldata signatures, uint256 threshold)
internal pure returns (bool) {
uint256 expectedLength = SafeModuleSignatures.signatureLength(signatures, thresh
return signatures.length == expectedLength;
}
```

목적:

- 악의적인 번들러의 서명 패딩 방지
- 불필요한 가스 소모 차단
- 정확한 서명 구조 강제

가스 한도 관리

```
uint256 public pbhGasLimit;
```

```
function setPBHGasLimit(uint256 _pbhGasLimit) external onlyOwner {
if (_pbhGasLimit == 0 || _pbhGasLimit > block.gaslimit) {
revert InvalidPBHGasLimit(_pbhGasLimit);
```

```
}

pbhGasLimit = _pbhGasLimit;

}
```

기능:

- PBH 트랜잭션의 최대 가스 사용량 제한
- 블록 가스 한도 내에서 동적 조정 가능
- DoS 공격 방지

6.7 에러 처리 및 예외 상황

주요 에러 타입

1. **InvalidNullifier**: Nullifier 중복 사용
2. **InvalidExternalNullifier**: 잘못된 외부 nullifier
3. **InvalidProofSize**: 증명 데이터 크기 오류
4. **UnauthorizedBuilder**: 승인되지 않은 빌더
5. **StorageCollision**: 임시 스토리지 충돌
6. **GasLimitExceeded**: 가스 한도 초과

예외 상황 처리

```
// 1. 월 경계 처리

function  verify(uint256 externalNullifier, uint16 numPbhPerMonth, uint256 signa

require(month == BokkyPooBahsDateTimeLibrary.getMonth(block.timestamp), "Invalid
```

```
// 월이 바뀌면 이전 월의 nullifier는 무효화됨

}



// 2. 업그레이드 중 상태 보존

uint256[49] private __gap;  // 스토리지 갭으로 안전한 업그레이드



// 3. 프록시 컨텍스트 검증

modifier  onlyProxy() {

require(address(this) != implementation, "Function must be called through delega

_;

}
```

이러한 포괄적인 분석을 통해 World Chain의 PBH 스마트 컨트랙트 시스템이 어떻게 복잡한 요구사항들을 안전하고 효율적으로 처리하는지 완전히 이해할 수 있습니다.